

Homework 2: Nondeterminism and Pumping Lemma

Due:02/05/2026

This assignment is due on **11:59 PM EST, Thursday, February 05, 2026**. You may turn it in up to 48 hours late, but assignments turned in by the deadline receive 3% extra credit. Additionally, late submission means late feedback, which means less time to study before an exam.

You should submit a typeset or *neatly* written pdf on Gradescope. The grading TA should not have to struggle to read what you've written; if your handwriting is hard to decipher, you will be asked to typeset your future assignments.

Enter your work using the `answer` environment we have provided for you. The default color is teal, but you can choose to change it to any of red, brown, magenta, violet, blue, darkgray, or black like so:

```
\begin{answer}[<COLOR>]
  <YOUR WORK GOES HERE>
\end{answer}
```

You may collaborate with other students, but any written work should be your own.

1. Nondeterminism (4 points)

For the following proofs, you must provide a construction of a finite automaton by describing its components to show that the language is regular.

(2 points - *Section A*) For any regular language L , prove that $L' = \{w \text{ where all } a\text{'s are removed} \mid w \in L, a \in \Sigma\}$ is regular.

Answer:

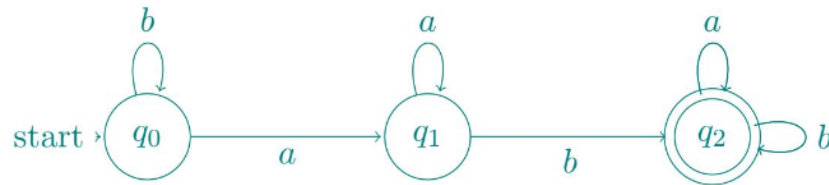
(2 points - *Both*) For any regular language L , prove that $L' = \{xay \mid xy \in L, a \in \Sigma, x, y \in \Sigma^*\}$ is regular. Notice that we form L' by taking the strings in L , and adding a single symbol anywhere in the string, even possibly at the beginning or end.

Answer:

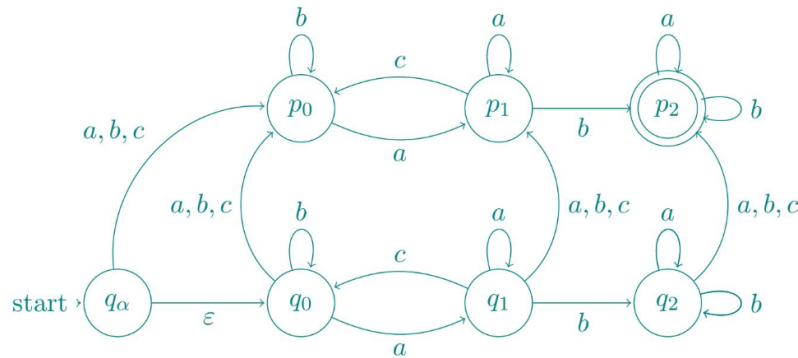
First let us denote the addition symbol as $\alpha \in \Sigma$. Now if L is regular then there exists some DFA that accepts it. Now every DFA is an NFA, so let's consider this to be some NFA, N , that accepts L . Now we make the following modification:

- (a) In our new NFA labeled M , we create two copies of N . If r_i denote the states of N , then p_i and q_i denote the new copies corresponding to r_i such that if δ is our new transition function and δ_0 was that of N , then we have $\delta(q_i, k) = \delta_0(r_i, k)$ and $\delta(p_i, k) = \delta_0(r_i, k)$. We also make sure to unmark the start and accept states in M except for that in p (i.e., if r_k is an accept state in N , then we unmark q_k as an accept state but p_k will still be an accept state).

- (b) We now add a new start state q_α such that $\delta(q_\alpha, \varepsilon) = q_1$ and $\delta(q_\alpha, \alpha) = p_1$ for all $\alpha \in \Sigma$.
- (c) Now for each q_i we have $\delta(q_i, \alpha) = p_i$ for every $\alpha \in \Sigma$.



In this case the modification we make is as follows,



Now we need to show that M accepts all strings in $\{xy : xy \in L, x, y \in \Sigma^*\}$. Consider two arbitrary strings $x, y \in \Sigma^*$ such that $xy \in L$. And take an arbitrary symbol $\alpha \in \Sigma$. Now note that as $x \in L$ the q states will simulate x as in the original DFA. Now given the symbol α , the NFA can choose to transition to the corresponding state in p , now as the states p are also a perfect simulation of the original DFA the string y will also be accepted. Hence, we show that any string of the form $x\alpha y$ such that $xy \in L, \alpha \in \Sigma, x, y \in \Sigma^*$ is accepted by M . On the other hand consider a string that is accepted by our automate. Now for a string to be accepted it needs to jump to the p states from the q states. Let a string before the jump be x , let the jump be α and let the string after the jump be y . Now note that if x ends at q_n then y begins at p_n as per our construction. But as the p states are a simulation of our original DFA, this means that we have xy a string that is accepted by the original DFA and hence we can write any string in the form $x\alpha y$ where $xy \in L$ and α some alphabet.

(2 points - Section X) Let L be a regular language, and let $L^\#$ be the set $\{x \in \Sigma^* \mid \text{for some } y \in L, y \text{ has the same number of 1's as } x\}$. Prove that, if L is regular, $L^\#$ is regular.

Answer: If L is regular, there is a DFA for it, and consider this to be the NFA of L called N . Now for each state $q \in Q$ of N , we modify δ such that $\delta(q, a) = q$ where $a \in \Sigma$

and $a \neq 1$ and for any a such that $\delta(q_0, a) = q_1$ where $q_0 \neq q_1$ we add $\delta(q_0, \varepsilon) = q_1$. The point of these two additions is that we allow each state to loop to itself any number of times as long as it's not a 1, as we only care about the counts of 1's. And the second modification allows us to skip transitions that are not 1 as again we don't want to enforce the other structures of L that do not count the 1.

Now assume that x is a string accepted by our new NFA. Now running down the computation for this string, at each state we have three choices. To self loop an alphabet other than 1, to ε transition (if it exists) or to take the transition like that of the original DFA. Now the first two options do not add or remove from the number of 1's (note that although it's possible for us to have an ε and 1 transition, the ε transition implies that in the original DFA it was also possible to skip the 1), so the third option would be to follow the transition of L in which case we preserve the count of 1. So we show the computation preserves the count of 1 to that of L and hence the string is in $L^\#$.

Now assume $x \in L^\#$ we need to show that x is a string accepted by our NFA. Now clearly x has the same number of 1's as in some string $y \in L$ (by definition of $L^\#$). Now this means that by adding or removing non-1's from y we can construct x . Let $y_0 \cdots y_n$ be our string, then after each y_i we can either add non-1's or for each $y_i \neq 1$ we can replace it with ε . So first note that y is trivially accepted by our NFA as we preserve the DFA of L . So for each y_i our new NFA is in some state q_i , now addition of non-1 states is allowed as we allow self loops for non-1's, and removal of $y_i \neq 1$ is allowed by allowing q_i to skip to q_{i+1} . Hence, our NFA is able to simulate any function that transforms $y \in L$ to $x \in L^\#$ and hence we have $x \in L^\#$.

Regular Expressions ***Both Sections*** (4 points)

For the following parts, you may assume $\Sigma = \{0, 1\}$.

(2 points) Give a regular expression for each of the following languages:

- The set of all strings with a multiple of three number of ones.

Answer:

$0^*(10^*10^*10^*)^*$

- The set of all strings where the number of 0's is equivalent to 2 mod 3.

Answer:

$1^*01^*0(01^*01^*0|1)^*$

(2 points) Give an equivalent NFA for the following regular expression:

$((11 \cup 01^*1 \cup 01)0)^*(11)^*$

Answer:

Claim: For any positive integer $n \geq 3$, there is a language whose minimal DFA contains n states.

Proof: For some $k \in \mathbb{Z}$, let $L_k = \{a^k\}$, a set with one element. Now, suppose D is a minimal DFA for L_k . You will prove that D has at least $k+2$ states. Let $\{q_i\}_{i=0}^k = q_0, \dots, q_k$ be the sequence of states (not necessarily distinct!) that D follows when reading a^k .

(1 point) Explain why $\{q_i\}_{i=0}^k$ cannot contain any repeated states (hint: the proof of the pumping lemma), and conclude that D has at least $k+1$ states.

Answer:

(1 point) Now consider the state that D ends in upon reading the string a^{k+1} , and argue that D must have at least $k+2$ states.

Answer:

(1 point) Demonstrate that D has exactly $k+2$ states by giving an explicit construction of D .

Answer:

(1 point) Complete the conclusion of this proof: "Therefore, we have given a constructive proof of the original claim. For any $n \in \mathbb{Z}, n \geq 3$, the language _____ has a minimal DFA with n states."

Hint: Your answer should be something, like L_{3n} , in terms of n .

Answer:

Polynomial Length ***Section X only*** (4 points)

Given $f(n)$, a polynomial with non-negative integral coefficients, let $L_f = \{1^{f(n)} \mid n \in \mathbb{N}\}$.

- For exactly which $f(n)$ is L_f regular? Prove your answer.

Answer: We know that our alphabet consists of just 1's and that we have a finite automata. First let's analyze all the possible automata that we can make under these conditions. As we only have one alphabet, each state has only one arrow going away from it or towards it. So we have two cases,

- a) We have no true loops in our automata. In this case this we have no loops at all (so no self loops either). But in this case the range i.e the language that is accepted is finite. Now note that given a polynomial with positive integral coefficients, if the coefficients of $n^k, k \geq 1$ are non-zero then f is increasing and hence has a countable range. So the only option is that all those coefficients are zero and our function is $f(n) = k$ where $k \in \mathbb{N}$.
- b) Second case is we have only ONE loop. Note that we CANNOT have more than one loop. Assume we have two loops for instance, then there must exist a state that has two transitions for the same alphabet (in this case 1) as we would need one to continue a loop and one to get one of the first loop. However, if we only have ONE loop, we do not need a transition to escape the loop.

Now this could either be a self loop or a loop with more than 1 state. In both cases consider the two divisions in the DFA, a finite length of states and a finite loop. With the finite length of states we can only have a finite range (and note we showed above this can only be generated by a constant) and the loop (say the loop is length k) will

generate values of the form $a_0 + kn$ where a_0 is some constant number of states we have to pass through before reaching the loop. So the possible functions are of the form $a_0 + a_1n$ where $n \geq 0$. So we've shown that all the DFAs will generate either of these two functions which means that any other polynomial that have non-zero coefficients for $n^2, \dots$ cannot form a regular language as we did above.

- Prove that for all other $f(n)$, L_f is not regular.

Answer: Clearly we showed above the every DFA that can be formed can be mapped onto a function of the form we described above. The contrapositive of this statement implies that for every function not of that form there does not exist a DFA that accepts it and hence is not regular.

Or another way of looking at it is, assume a polynomial f that is not linear in n is such that 1^f is a regular language i.e has a DFA. Now using the pumping lemma let p be the pumping length and let w be some string greater than the pumping length. We write $w = xyz$ where $|y| \geq 1$ and $|xz| < p$. Note now that any string of the form xy^iz is accepted by the language. Let $|y| = k$ this must then mean that if $|w| = h$ then the strings $1^h, 1^{h+k}, 1^{h+2k}, \dots$ are all accepted by the DFA. But note now that if f is non-linear then it cannot produce numbers of the form $h, h+k, h+2k, h+3k, \dots$ as f must increase exponentially or in other words if $n > m$ then $f(n+1) - f(n) > f(m+1) - f(m)$ (note it's strictly greater). But this is a contradiction as we just showed that we have a difference of k for each of the values.